

Rigtorp MPMC Queue: algoritmo, port Mojo e valutazione in MoStream

Documentazione tecnica per attività di tesi

Progetto MoStream - 22 luglio 2026

Questa relazione descrive il port Mojo della bounded MPMCQueue turn-based di Erik Rigtorp. La coda assegna ticket mediante Fetch-And-Add, associa ogni ticket a uno slot e a una generazione, e usa un contatore atomico turn per coordinare producer e consumer. Il port isola contatori globali e slot su cache line da 64 byte. Nel primo confronto con CAS-MPMC, FAA compatta e Hybrid, Rigtorp ottiene il miglior throughput medio in 10 configurazioni su 18, dominando tutti i workload sbilanciati 1P-8C e 8P-1C. Non è però universalmente superiore: FAA prevale ad alta contesa bilanciata e la successiva Padded-FAA risulta la migliore scelta generale. Rimangono inoltre ticket irrevocabili, head-of-line blocking, puro spin e un footprint di memoria elevato.

Scopo e provenienza

Il file MoStream/Rigtorp_queue.mojo è dichiarato come port Mojo dell'implementazione bounded MPMCQueue di Erik Rigtorp, distribuita con licenza MIT. La relazione analizza il codice presente nel repository MoStream; non costituisce una prova formale di equivalenza con ogni revisione del sorgente C++ originale.

MPMC significa Multiple Producer, Multiple Consumer: più thread possono inserire e rimuovere elementi concorrenti. Bounded significa che la memoria del ring buffer viene allocata alla costruzione e la capacità fisica non cresce. Nel port la capacità deve essere almeno uno e può essere arbitraria. Il benchmark comune usa soltanto potenze di due, perché le altre code confrontate impongono questo vincolo.

La coda è progettata per push e pop bloccanti ad alto throughput. Quando il ring è pieno, push attende; quando è vuoto, pop attende. Le varianti try_push e try_pop ritornano senza attendere se l'operazione non può essere completata.

Struttura dati

I campi principali di RigtorpMPMCQueue sono:

slots	array di RigtorpSlot[T]
capacity	numero fisso di celle del ring
head	prossimo ticket riservabile da un producer
tail	prossimo ticket riservabile da un consumer

head e tail sono PaddedAtomicU64 distinti. Ogni oggetto contiene un Atomic UInt64 e byte di padding fino a 64 byte. L'obiettivo è impedire che gli aggiornamenti dei producer su head invalidino la cache line modificata dai consumer su tail.

Ogni RigtorpSlot contiene:

turn	contatore atomico di stato e generazione
data	payload Optional[T]
padding	byte fino al successivo multiplo di 64

L'array è allocato con alignment=64. La dimensione di ogni slot è arrotondata a

un multiplo di 64 byte; quindi il primo slot e quelli successivi iniziano su un confine di cache line. Se turn e payload richiedono più di 64 byte, la cella occupa il multiplo successivo.

Ticket, indice e ciclo

Ogni operazione bloccante riceve un ticket monotono t . Il ticket individua sia la posizione fisica sia la generazione logica:

```
index(t) = t modulo capacity  
cycle(t) = t div capacity
```

L'indice riutilizza lo stesso ring; il ciclo distingue passaggi diversi sullo stesso slot. Questa scelta supporta capacità non power-of-two, a differenza della mask ticket $\& (capacity-1)$, ma richiede modulo e divisione intera.

Protocollo del turn

Per il ticket t e il ciclo $g = cycle(t)$, il producer attende il valore $2g$ e il consumer attende $2g+1$:

```
tab( ), box;  
Turn Significato Attore abilitato  
2g slot libero nel ciclo g producer  
2g+1 payload pubblicato nel ciclo g consumer  
2g+2 slot libero nel ciclo g+1 producer successivo
```

Il turn parte da zero. Dopo la scrittura, il producer lo incrementa di uno; dopo la lettura, il consumer lo incrementa ancora. Il valore crescente elimina l'ambiguità fra slot vuoto e slot pieno e associa lo stato alla corretta generazione del ring.

Esempio con capacità quattro: i ticket 0 e 4 usano entrambi lo slot zero. Il ticket 0 attende turn 0, pubblica turn 1 e il consumer libera con turn 2. Il ticket 4 appartiene al ciclo uno, attende turn 2 e pubblica turn 3.

Push bloccante

Il percorso di inserimento è:

```
t = head.fetch_add(1, SEQ_CST)  
s = slots[t modulo capacity]  
g = t div capacity  
attendi finché s.turn.load(ACQUIRE) == 2g  
s.data = item  
s.turn.store(2g + 1, RELEASE)
```

Fetch-And-Add assegna un ticket distinto a ogni producer senza un ciclo di CAS sul contatore condiviso. Se il ring è pieno, il producer ha comunque già prenotato la propria posizione futura e attende il turn corretto. Il ticket non può essere annullato.

La store RELEASE sul turn pubblica il payload. Un consumer che osserva il turn con ACQUIRE vede la scrittura su data, che non deve quindi essere atomica.

Pop bloccante

Il percorso di estrazione è simmetrico:

```
t = tail.fetch_add(1, SEQ_CST)  
s = slots[t modulo capacity]  
g = t div capacity  
attendi finché s.turn.load(ACQUIRE) == 2g + 1
```

```
item = s.data.take()
s.turn.store(2g + 2, RELEASE)
```

La store finale libera la cella per il producer del ciclo successivo. Anche il consumer prenota prima di sapere quando il dato sarà disponibile, quindi un pop su coda vuota rimane in spin sul proprio ticket.

Memory ordering

Nel port corrente i fetch_add su head e tail usano Ordering.SEQUENTIAL, in linea con le operazioni predefinite seq_cst menzionate nel commento del codice originale. I turn usano load ACQUIRE e store RELEASE. Le operazioni try usano load ACQUIRE e compare_exchange con successo SEQUENTIAL e fallimento ACQUIRE.

La relazione di pubblicazione essenziale è stabilita dal turn: scrittura del payload prima della store RELEASE, poi osservazione tramite load ACQUIRE. È possibile studiare ordinamenti più deboli sui soli contatori di ticket, come fa Padded-FAA, ma una modifica del memory ordering richiede validazione specifica e non è implicita nei risultati qui riportati.

try_push e try_pop

Le operazioni non bloccanti non possono consumare alla cieca un ticket FAA: se lo slot non fosse pronto, abbandonare il ticket creerebbe un buco permanente. Per questo usano CAS.

try_push carica head, verifica il turn atteso e tenta compare_exchange da t a t+1. In caso di collisione, il valore expected aggiornato diventa il nuovo ticket e il ciclo prosegue. Se lo slot non è disponibile, rilegge head: ritorna il payload al chiamante soltanto quando head è rimasto invariato, riducendo i falsi fallimenti dovuti a una vista obsoleta.

try_pop applica lo stesso schema a tail. Restituisce Optional[T]: Some contiene l'elemento estratto, None indica che non era disponibile. try_push usa la convenzione complementare: None indica inserimento riuscito, mentre Some restituisce al chiamante il payload non inserito.

API Mojo e ciclo di vita

Il tipo richiede un payload Copyable ed è Movable:

```
from MoStream.Rigtorp_queue import RigtorpMPMCQueue

var queue = RigtorpMPMCQueue[Int](1024)
queue.push(42)
var value = queue.pop()
```

Metodi esposti:

- push(item): inserimento bloccante;
- pop(): estrazione bloccante;
- try_push(item): inserimento immediato con restituzione dell'item al fallimento;
- try_pop(): estrazione immediata tramite Optional;
- estimated_len(): differenza istantanea fra head e tail.

Il costruttore inizializza tutti i turn a zero. Il move constructor trasferisce i puntatori e conserva i contatori. Il distruttore distrugge ogni slot e libera l'allocazione allineata.

estimated_len non è una misura esatta dell'occupazione: head e tail contano anche ticket già riservati da operazioni ancora in attesa. Con producer o consumer prenotati in anticipo può superare la capacità o diventare negativo.

Deve essere usato solo come indicatore diagnostico, non per la correttezza.

Boundedness e uso della memoria

Il numero di payload materialmente contenuti non supera capacity. Possono però esistere più thread in attesa con ticket futuri; queste prenotazioni non espandono il ring.

Con slot minimi da 64 byte, la sola area delle celle richiede almeno:

```
tab( ), box;  
Capacità Memoria minima slot Osservazione  
16 1 KiB footprint ridotto  
1024 64 KiB ordine di grandezza delle cache private  
65536 4 MiB maggiore pressione su cache e TLB
```

A questi valori si aggiungono i contatori padded, i puntatori e l'eventuale arrotondamento dovuto al payload. Una FAA compatta può collocare più sequence number nella stessa cache line e consumare molto meno spazio; Rigtorp scambia memoria per minore false sharing.

False sharing e comportamento della cache

Senza padding, thread che operano su slot logici diversi possono modificare atomiche presenti nella stessa linea. Il protocollo di coerenza trasferisce o invalida l'intera cache line, creando contesa non richiesta dall'algoritmo. Rigtorp separa i turn degli slot e i contatori head/tail.

Il beneficio dipende dal workload, dalla capacità, dal payload e dalla microarchitettura. Per ring molto grandi, il padding può ridurre la località e aumentare miss e pressione TLB. I risultati sperimentali mostrano correlazioni, ma non isolano causalmente padding, modulo, memory ordering e politica di spin.

Politica di attesa

Il port usa puro spin: i cicli di push e pop rileggono turn senza sleep, yield o backoff. Su core dedicati questo può ridurre la latenza di risveglio. In oversubscription o durante attese lunghe può invece consumare un intero core e ritardare proprio il thread che deve far avanzare il turn.

FAA e Padded-FAA del repository effettuano uno yield periodico dopo un numero di spin. Il confronto di throughput include quindi anche la diversa politica di attesa; non misura soltanto la struttura turn-based.

Correttezza verificata

Il test Tests/test_rigtorp_queue.mojo usa capacità otto e verifica:

- riempimento completo tramite try_push;
- rifiuto di un elemento aggiuntivo a coda piena;
- ordine FIFO degli otto elementi;
- risposta vuota di try_pop dopo lo svuotamento.

Il test è stato rieseguito il 22 luglio 2026 con Mojo 1.0.0b1 e ha prodotto:

```
PASS: RigtorpMPMCQueue sequential correctness
```

La correttezza concorrente è controllata dal benchmark: ogni producer inserisce un intervallo disgiunto di interi e l'ultimo producer inserisce un sentinel per consumer. I consumer mantengono conteggio e checksum locali. Dopo il join si verificano numero totale e somma $N(N-1)/2$. Tutti i campioni usati nel CSV results_with_rigtorp.csv hanno restituito valid=True.

Questi test rilevano perdita, duplicazione e molte anomalie di concorrenza, ma non sostituiscono una prova formale di linearizzabilità né un model checker.

Metodologia del benchmark

La campagna principale usa:

- topologie 1P-1C, 2P-2C, 4P-4C, 8P-8C, 1P-8C e 8P-1C;
- capacità 16, 1024 e 65536;
- 250000 messaggi per producer;
- 10 ripetizioni per configurazione;
- rotazione dell'ordine di esecuzione per ridurre il bias d'ordine;
- media, deviazione standard e intervallo di confidenza 95 percento con Student-t.

Ambiente rilevato: Intel Xeon Gold 5512U, 28 core e 56 thread hardware, un nodo NUMA, Linux 5.15.0-168-generic, x86-64 e Mojo 1.0.0b1.

Risultati: confronto iniziale

La tabella usa milioni di messaggi al secondo. Best è il miglior valore medio fra CAS, FAA, Rigtorp e Hybrid con soglie 1, 2, 4 e 8 nella campagna results_with_rigtorp.csv.

```
tab( ), box;
Topologia Capacità CAS FAA Rigtorp Best
1P-1C 16 3.58 5.23 5.11 FAA
1P-1C 1024 4.45 6.63 4.86 Hybrid-2
1P-1C 65536 4.56 6.32 2.83 FAA
2P-2C 16 4.47 6.58 7.21 Rigtorp
2P-2C 1024 5.88 9.67 8.77 FAA
2P-2C 65536 5.86 10.32 11.17 Rigtorp
4P-4C 16 3.70 8.55 8.46 FAA
4P-4C 1024 3.50 8.50 8.69 Rigtorp
4P-4C 65536 4.13 9.70 9.89 Rigtorp
8P-8C 16 2.67 9.93 8.49 FAA
8P-8C 1024 2.76 8.87 8.45 FAA
8P-8C 65536 2.90 9.08 8.53 FAA
1P-8C 16 3.52 4.12 7.16 Rigtorp
1P-8C 1024 3.44 4.25 6.74 Rigtorp
1P-8C 65536 3.87 4.53 7.65 Rigtorp
8P-1C 16 3.75 5.32 7.63 Rigtorp
8P-1C 1024 3.76 5.20 7.10 Rigtorp
8P-1C 65536 3.94 5.63 7.42 Rigtorp
```

Rigtorp ha il valore medio più alto in 10 casi su 18, FAA in sette e Hybrid-2 in uno. Il conteggio delle vittorie è descrittivo: quando gli intervalli di confidenza si sovrappongono, una differenza piccola non prova una superiorità statistica.

Interpretazione per topologia

Workload sbilanciati

Rigtorp vince tutte le sei configurazioni sbilanciate. In 1P-8C raggiunge da 1.59 a 1.74 volte il throughput FAA; in 8P-1C raggiunge da 1.32 a 1.44 volte FAA. L'isolamento dei turn e il percorso a ticket risultano particolarmente efficaci quando la contesa si concentra su un solo lato della coda.

Il benchmark non separa il contributo del padding dal puro spin o dagli ordinamenti; la spiegazione va quindi formulata come ipotesi coerente con i dati, non come prova causale.

Contesa bilanciata media

In 2P-2C e 4P-4C Rigtorp e FAA sono spesso vicine. Rigtorp prevale su FAA con capacità 65536 e in due casi con capacità 1024 o 16. Per 2P-2C capacità 16 gli intervalli 95 percento sono molto ampi e sovrapposti; la classifica media non è sufficiente per dichiarare un vincitore robusto.

Contesa bilanciata elevata

In 8P-8C FAA compatta vince tutte le capacità. Rigtorp raggiunge circa l'85, 95 e 94 percento del throughput FAA rispettivamente con capacità 16, 1024 e 65536. Il padding non compensa sempre modulo, divisione, seq_cst e footprint.

Un producer e un consumer

In 1P-1C Rigtorp non è la scelta migliore. Con capacità 65536 scende a 2.83 Mmsg/s contro 6.32 di FAA. In assenza di contesa MPMC, memoria aggiuntiva e operazioni generali sono costi difficili da ammortizzare; una vera SPSC può eliminare del tutto FAA e CAS sui contatori posseduti da un solo thread.

Confronto con Padded-FAA

Una campagna successiva, salvata in results_padded_faa.csv, aggiunge Padded-FAA: ticket FAA, slot isolati come Rigtorp, indice tramite mask, contatori RELAXED e yield periodico. Non bisogna mescolare numericamente run distinte, ma il risultato qualitativo cambia: Padded-FAA è prima in 13 configurazioni su 18 ed è la migliore scelta generale osservata.

Rigtorp conserva un vantaggio specifico in 1P-8C capacità 16, con 7.91 Mmsg/s contro 7.62 di Padded-FAA. In 1P-8C capacità 1024 le due sono vicine, 7.78 e 7.98 Mmsg/s. Padded-FAA prevale nei workload bilanciati da 2P-2C in su e in gran parte dei workload 8P-1C.

Rigtorp rimane quindi un'importante baseline e una scelta competitiva per workload sbilanciati, ma non è migliore di tutte le code considerate dopo l'introduzione di Padded-FAA.

Confronto qualitativo

```
tab( ),box;
Proprietà CAS-MPMC FAA compatta Hybrid Rigtorp
Prenotazione bloccante CAS FAA CAS poi FAA FAA
Slot padded No No No Sì
Indice mask mask mask modulo
Ticket ordering RELAXED RELAXED RELAXED SEQ_CST
Attesa spin spin/yield spin/yield puro spin
Capacità arbitraria No No No Sì
Ticket irrevocabile No Sì nel fallback Sì
Memoria slot compatta compatta compatta >=64 byte
```

CAS-MPMC mantiene la possibilità di attendere senza prenotare un ticket futuro, ma soffre retry sotto contesa. FAA compatta riduce le collisioni e la memoria. Hybrid prova a usare CAS come fast path e FAA dopo collisioni, pagando entrambi i costi quando il fallback è frequente. Rigtorp privilegia isolamento e throughput a costo di memoria e spinning.

Garanzie di progresso e head-of-line blocking

Il port non è wait-free. Dopo fetch_add un thread possiede un ticket

irrevocabile. Se viene sospeso prima di pubblicare o liberare il proprio slot, le operazioni associate a turn successivi possono rimanere bloccate anche se altri core continuano a eseguire.

```
thread A riserva ticket t
thread A viene sospeso prima dello store del turn
thread B raggiunge un ticket successivo sullo stesso ordine logico
thread B non può scavalcare il turn mancante
```

Questo head-of-line blocking è lo stesso limite concettuale delle varianti FAA bloccanti. L'ordine dei ticket offre un ordine di servizio logico, ma non garantisce fairness temporale se lo scheduler ritarda un proprietario.

Il puro spin aggrava il rischio in oversubscription: i waiter possono occupare i core necessari al thread che deve completare la pubblicazione. Non è stato eseguito un esperimento mirato che sospenda un thread immediatamente dopo FAA; questa rimane una validazione necessaria prima di usare la coda in componenti sensibili al progresso, come alcune queue dello scheduler.

Altri limiti

- Il consumo di memoria è almeno una cache line per slot.
- Modulo e divisione possono costare più di una mask power-of-two.
- Puro spin aumenta consumo CPU ed energia durante le attese.
- try_push e try_pop tornano al CAS e non beneficiano direttamente del FAA.
- I contatori UInt64 hanno un wrap-around teorico non studiato nei test.
- Non sono state misurate latenza p50/p95/p99, fairness o energia.
- I risultati provengono da una singola macchina e una versione beta di Mojo.
- Il benchmark usa payload Int; payload grandi possono cambiare cache e memoria.
- Non è inclusa una prova formale del memory model o della linearizzabilità.

Indicazioni per MoStream

Rigtorp è adatta come implementazione opzionale dei Communicator quando la topologia è fortemente sbilanciata e i worker dispongono di core sufficienti. È meno indicata come default universale, perché perde in 1P-1C, in 8P-8C e nel confronto complessivo con Padded-FAA.

Scelta pratica derivata dai dati:

1. usare una SPSC dedicata per 1P-1C;
2. considerare Rigtorp o una SPMC/MPSC padded specializzata per 1P-NC e NP-1C;
3. preferire Padded-FAA come MPMC generale orientata al throughput;
4. mantenere CAS o Hybrid dove sospensione e ticket irrevocabili sono critici;
5. selezionare il tipo concreto alla costruzione, evitando un branch runtime per ogni messaggio.

Esperimenti futuri

1. Ripetere su macchine isolate, architetture diverse e più nodi NUMA.
2. Misurare p50, p95, p99, fairness, utilizzo CPU ed energia.
3. Sospendere deliberatamente producer e consumer subito dopo fetch_add.
4. Confrontare puro spin, pause instruction, spin-then-yield e parking.
5. Separare sperimentalmente padding, modulo e memory ordering.
6. Provare capacità non power-of-two, caratteristica specifica di Rigtorp.
7. Provare payload di dimensioni differenti e ring oltre la LLC.
8. Integrare la coda in pipeline MoStream reali, non solo microbenchmark.
9. Implementare SPMC e MPSC specializzate senza RMW sul lato a singolo thread.
10. Applicare model checking o test sistematici degli interleaving.

Conclusione

Il port Rigtorp è una bounded MPMC ad alte prestazioni fondata su ticket FAA, turn per generazione e isolamento delle cache line. Nel confronto iniziale è la migliore media in 10 configurazioni su 18 e domina i workload sbilanciati. Il risultato sostiene la sua applicabilità a canali MoStream 1P-NC e NP-1C.

Non è però una coda universalmente dominante. FAA compatta è migliore ad alta contesa bilanciata, Padded-FAA ottiene il miglior risultato complessivo nella campagna successiva e SPSC resta la direzione corretta per 1P-1C. Head-of-line blocking, puro spin e memoria impediscono inoltre di dedurre la scelta soltanto dal throughput.

Formulazione sintetica per la tesi: Rigtorp dimostra che l'isolamento degli slot su cache line può migliorare fortemente i workload MPMC sbilanciati; il suo vantaggio è tuttavia dipendente dalla topologia e deve essere valutato insieme a progresso, latenza, CPU e footprint.

Artefatti e riferimenti

- MoStream/Rigtorp_queue.mojo, port Mojo analizzato.
- Tests/test_rigtorp_queue.mojo, test sequenziale.
- Benchmarks/QueueBenchmark/queue_benchmark.mojo, benchmark concorrente.
- Benchmarks/QueueBenchmark/run_suite.py, runner e statistica.
- Benchmarks/QueueBenchmark/results_with_rigtorp.csv, campagna principale.
- Benchmarks/QueueBenchmark/results_padded_faa.csv, confronto successivo.
- Erik Rigtorp, MPMCQueue, <https://github.com/rigtorp/MPMCQueue>, licenza MIT.